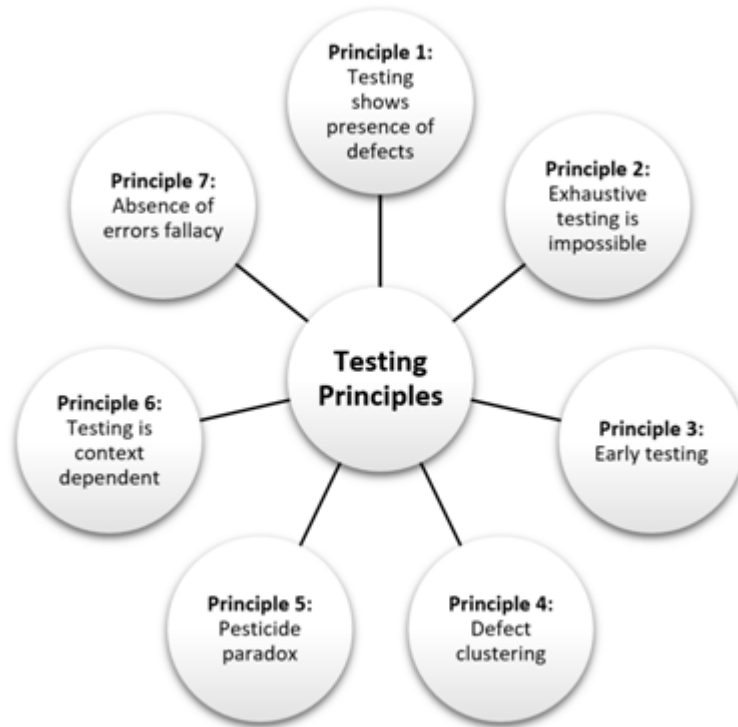


Software Testing 301 - Final Certification Activity

1. Testing principles



Principle 1: Testing Shows Presence of Defects

The purpose of testing is to discover bugs in software but no matter how much testing we carry out or if no bugs are discovered, we cannot say that there are no defects in the system. Testing will reduce the probability of any hidden bugs remaining in the software however it is not a proof that the software is bug free.

Real Time Example: During the testing of a web application, if a user encounters a broken link or an error message, it indicates the presence of a defect.

Principle 2: Exhaustive Testing is Impossible

As a tester, one of the most important concepts to be aware of is that it's not possible to test everything i.e. testing all the possibilities, although you might be tempted to. When testing, you have to keep in mind that software development projects work on budgets and constraints. Testing all the scenarios will require more resources therefore will require more time and more money. This would lead to going over budget and over time. This in turn will cause project failure.

To combat this, we need to determine where to focus our testing effort based on risks and priorities. Testing should also provide good coverage of the entire project rather than focusing all the testing effort in one specific area. Create a test plan according to the risks, priorities and coverage given the budget and time. This will lead to more effective and higher quality testing.

Real Time Example: Testing a mobile app on every device model, operating system version, and screen size is impractical due to the large number of combinations.

Principle 3: Early Testing

The earlier the bugs are found, the less the cost to fix them. Bugs found later in the development stage are expensive to fix. The following graph shows the cost of the project compared to the time the bugs are found.

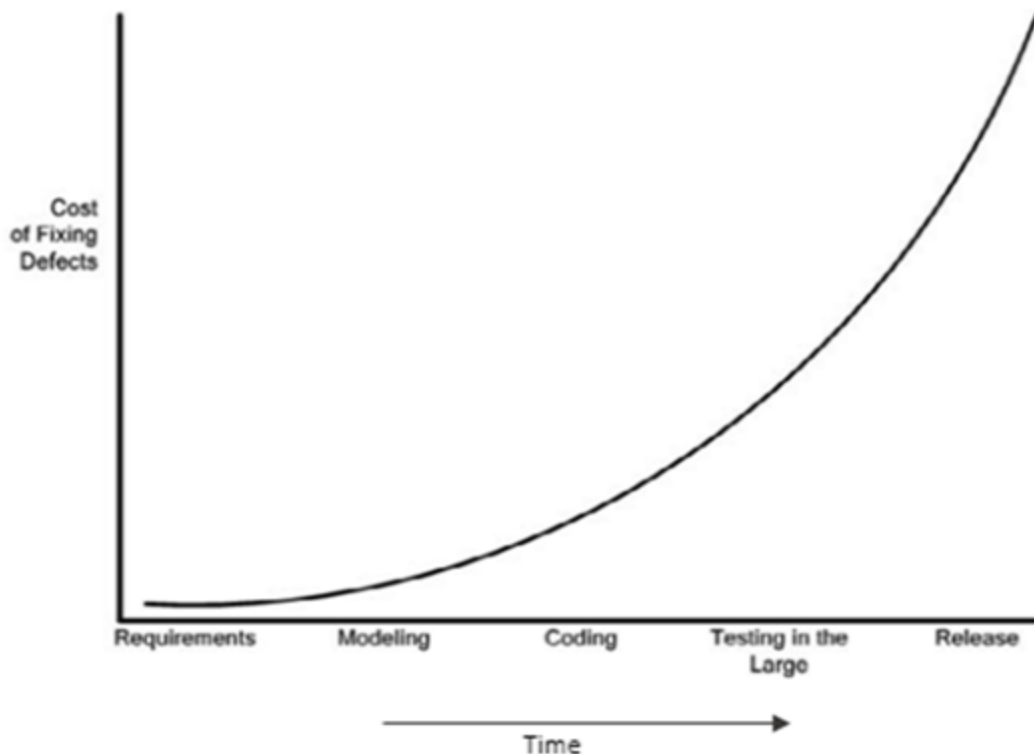


Figure 1: The rising costs of finding and fixing defects. (Ambler, 2002)

Therefore, it is best to start testing as early as possible in the software development lifecycle (SDLC). Conduct early test design, requirements and reviews to capture any gaps and fix them prior to starting development work. It is a good idea to involve the test team right from the beginning of the SDLC so they have a good understanding of the product and can create more effective tests.

Real Time Example: Unit tests are performed on individual modules of a software application as soon as they are developed to detect and fix defects early on.

Principle 4: Defect Clustering

This means that defects are not distributed evenly across an application and tend to concentrate or cluster in a few areas. Thus, a few modules of the application will contain the most number of defects. We can apply the Pareto Principle, (the 80-20 rule) which says that 80% of the bugs are in 20% of the code or modules.

This information can be used during risk assessment for test planning to allocate more time for testing areas with known complex code/sections of the application. Testing these complex areas early can help to reduce the time and cost of finding defects.

Also if you come across a defect during testing, it is likely there will be more in that area. The cause of these clusters is because some areas of the code can be complex and tricky or making changes can have a rippling effect in the software and create more defects.

Real Time Example: A banking software module responsible for interest calculations consistently contains more defects compared to other modules.

Principle 5: Pesticide Paradox

This principle says that if the same set of test cases are repeated over and over again, it will no longer find any new bugs. This is similar to insects building up resistance to pesticides over time. The reason for this is when bugs are initially found and fixed, the number of bugs will reduce as the software has improved hence the same tests will not find any new bugs as the defect clusters change over time. To prevent pesticide paradox, testing now needs to focus on other areas or risks such as looking at the requirements and design documents for defects, writing new test cases to test different areas of the system that were not tested previously and writing test cases to test different paths or using different test data.

Writing more test cases and testing them will require more testing time hence increase the cost. Therefore it is important to work according to the priority of test cases. Review and update your test cases depending on changes to the product, new releases and remove any unwanted tests. This will help in managing the testing time.

As the 'hot spots' for bugs get cleared up, we need to move our focus elsewhere, to the next set of risks. Over time, our focus may change from finding coding bugs, to looking at the requirements and design document defects, to looking for process improvements to minimize bugs in the software.

Real Time Example: Repeating the same login test with the same set of credentials on a website does not reveal new defects or vulnerabilities.

Principle 6: Testing is Context Dependent

Testing needs to be tailored according to the type of project. For example, testing an air traffic control system should be different to testing an online shopping website.

Different software systems have different levels of risk and impact therefore testing needs to reflect this. More focus and more rigorous testing will need to be carried out for an air traffic control system than the online shopping website because it has a much higher risk and impact if defects were found in the software compared with the online website.

In some projects, a spelling error can seem trivial. However for another project this could have a major impact on the reputation of the business. Hence, when presented with a project, you should first recognise the risks and impact of the project in order to determine the different methodologies, techniques and the types of testing to apply. This will help to provide a quality product and manage the testing time and budget.

Real Time Example: Testing a medical device requires adherence to strict regulatory standards compared to testing a gaming application.

Principle 7: Absence of Error Fallacy

It may seem that the software has been tested and all the defects have been fixed.. However the system could still end up being unusable by the client. This can occur when the software developed did not meet the business needs and requirements of the user.

The main purpose of software is to assist the user in completing their tasks effectively and efficiently. Hence when conducting testing activities, it is important to keep the user needs and requirements in mind. Test cases should not be developed for the wrong requirements.

It's important to have reviews and meetings to discuss the requirements with the whole team.

It may seem like these principles are just theory and will not really be used in practice when carrying out testing work. However this is not the case. Keeping these principles in mind will help to plan testing activities and test software diligently in a methodical manner that can save time and money. If the principles seem too daunting at first, you can start applying one principle at a time and slowly work through the rest. Through experience, you will automatically apply these principles in your thought processes when testing.

Real Time Example: A website appears to function correctly during testing but still has usability issues that impact user experience.

2. Levels of testing

Software testing is a critical part of the software development life cycle (SDLC) that ensures the quality and reliability of the software. There are different levels of software testing, each with its own purpose and scope. Here are the common levels of software testing along with real-time examples:

Unit Testing:

Unit testing focuses on testing individual components or units of code to ensure they work correctly in isolation. Developers often perform unit testing during the development process.

Real Time Example: A developer might test a function that calculates the total price of items in a shopping cart to ensure it returns the correct value.

Integration Testing:

Integration testing verifies the interaction between different components or modules to ensure they work together as expected. It checks if the integrated units communicate and share data correctly.

Real Time Example: Testing the integration between a frontend application and a backend API to ensure data is exchanged accurately.

System Testing:

System testing involves testing the entire system as a whole, including all integrated components, to validate that it meets the specified requirements.

Real Time Example: Testing a web application to ensure all functionalities, such as user registration, login, and data retrieval, work together seamlessly.

Acceptance Testing:

Acceptance testing aims to determine if the system meets the requirements and satisfies the end-user or customer. It can be performed by the end-users or a separate testing team.

Real Time Example: A customer might conduct acceptance testing on a mobile app to ensure it meets their expectations and performs as intended.

Regression Testing:

Regression testing is conducted when changes or enhancements are made to the software to ensure that the existing functionalities haven't been affected.

Real Time Example: For instance, after fixing a bug in a software application, regression testing is performed to ensure that the fix didn't introduce any new issues.

Performance Testing:

Performance testing assesses the performance, scalability, and stability of the software under various conditions, such as high user loads or heavy data processing.

Real Time Example: Testing a website with a large number of concurrent users to determine its response time and resource utilization.

Security Testing:

Security testing focuses on identifying vulnerabilities and weaknesses in the software to ensure it is secure against unauthorized access, data breaches, and other security risks.

Real Time Example: Conducting penetration testing to simulate attacks and identify potential security loopholes.

Usability Testing:

Usability testing evaluates how user-friendly and intuitive the software is for end-users. Testers observe users performing tasks and gather feedback to identify areas for improvement. For instance, conducting usability testing on a mobile app to assess how easily users can navigate and perform actions.

Real Time Example: if you're testing a mobile app, you might observe users interacting with the app, gather feedback on their experience, and make improvements based on their suggestions.

3. Decision Table Testing

Decision table testing is a software testing technique used to test system behavior for different input combinations. This is a systematic approach where the different input combinations and their corresponding system behavior (Output) are captured in a tabular form. That is why it is also called as a Cause-Effect table where Cause and effects are captured for better test coverage.

A Decision Table is a tabular representation of inputs versus rules/cases/test conditions. It is a very effective tool used for both complex software testing and requirements management. A decision table helps to check all possible combinations of conditions for testing and testers can also identify missed conditions easily. The conditions are indicated as True(T) and False(F) values.

Real Time Example: How to create a decision table for a login screen

 **Let's Get Started** 

[Forgot ?](#)

Login

Don't have an account? [Sign Up](#)

The condition states that if the user provides the correct username and password the user will be redirected to the homepage. If any of the input is wrong, an error message will be displayed.

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Username	F	T	F	T
Password	F	F	T	T
Output	E	E	E	H

In the above example,

- T – Correct username/password
- F – Wrong username/password

- E – Error message is displayed
- H – Home screen is displayed

Now let's understand the interpretation of the above cases:

- Case 1 – Username and password both were wrong. The user is shown an error message.
- Case 2 – Username was correct, but the password was wrong. The user is shown an error message.
- Case 3 – Username was wrong, but the password was correct. The user is shown an error message.
- Case 4 – Username and password both were correct, and the user is navigated to the homepage.

4. Use Case Testing

Use Case Testing is a software testing technique that helps to identify test cases that cover entire system on a transaction by transaction basis from start to end. Test cases are the interactions between users and software application. Use case testing helps to identify gaps in software application that might not be found by testing individual software components.

A Use Case in Testing is a brief description of a particular use of the software application by an actor or user. Use cases are made on the basis of user actions and the response of the software application to those user actions. It is widely used in developing test cases at system or acceptance level.

Real Time Example: Let's consider a scenario where a company is developing an e-commerce website. One of the key functionalities of the website is the shopping cart feature. Use case testing can be applied to validate the shopping cart functionality.

Use Case: Add Product to Shopping Cart

Use Case Description: The user adds a product to the shopping cart.

Precondition: The user is logged in to the e-commerce website.

Main Flow:

- a. The user browses the website and selects a product.
- b. The user clicks on the "Add to Cart" button.
- c. The system validates the product availability and adds it to the shopping cart.

- d. The system displays a success message confirming the product has been added to the cart.
- e. The user can continue shopping or proceed to checkout.

Alternative Flows:

- If the product is out of stock:
 - The system displays an appropriate error message indicating the product is not available for purchase.
- If the user is not logged in:
 - The system redirects the user to the login page and prompts them to log in before adding the product to the cart.
- If there are any technical issues during the process:
 - The system displays an error message informing the user to try again later or contact customer support.

Use Case Testing Example:

Let's assume the following test scenario for the "Add Product to Shopping Cart" use case:

Test Scenario: Adding a Product to the Shopping Cart

Test Steps:

1. Open the e-commerce website and log in with valid credentials.
2. Search for a specific product.
3. Verify that the product details are displayed correctly.
4. Click on the "Add to Cart" button.
5. Check if the product is successfully added to the cart.
6. Verify that a success message is displayed confirming the product has been added.
7. Repeat steps 2-6 with different products, including scenarios with out-of-stock products.
8. Test adding a product to the cart without being logged in and ensure redirection to the login page occurs.
9. Test the process under high load conditions to ensure scalability and performance.

By conducting use case testing, the development team can validate that the shopping cart functionality of the e-commerce website works as intended and handles different scenarios accurately. It helps ensure a smooth and seamless shopping experience for users.

